33. Entegrasyon ve E2E Testleri: Playwright ile Uçtan Uca Doğrulama

Tüm Parçaların Bir Araya Geldiğinde Doğru Çalıştığını Kanıtlamak
Bölüm 6 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 13 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Tüm unit testler yeşil yanarken (başarılıyken) bir web sitesinde 'Giriş Yap' butonu tıklandığında neden hiçbir şey olmaz? Entegrasyon testleri neden bu kör noktayı çözer?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 200 adet mükemmel çalışan birim test, iki modülün birbiriyle konuştuğu sınırda (interface) hata olmayacağını garanti etmez. - 📌 Playwright, modern web uygulamalarını gerçek Chromium, Firefox ve WebKit tarayıcılarında headless olarak test eden altın standarttır. - 📌 E2E testleri sadece en kritik 'kullanıcı yolculuklarına' (User Journey - Kayıt, Ödeme, Giriş) uygulanmalıdır.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

İki mühendisin harika bir ev inşa ettiğini düşünün:

Biri dünyanın en kaliteli kapı kilidini üretti (Unit Test %100 başarılı). Diğeri ise dünyanın en sağlam ahşap kapısını yaptı (Unit Test %100 başarılı).

Fakat kapıyı yerine monte ettiklerinde bir baktılar: Kilit solda, kilit deliği sağda! Kapı kilitlenemiyor! İşte bu bir **Entegrasyon Hatasıdır**.

Birim testler parçaların tek başına sağlam olduğunu gösterir. **Entegrasyon ve E2E (Playwright) testleri** ise kapıyı duvara takıp anahtarı çeviren, kolu indiren ve kapının gerçekten kilitlendiğini gören nihai testtir.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Test Kapsamının Gerçeklik Seviyesi:

```
[Birim Test (Jest)]          —> "Fonksiyon matematiksel olarak doğru mu?"
            |
            ▼
[Entegrasyon Testi (Supertest)] —> "Express rotası veritabanına gerçekten yazıyor mu?"
            |
            ▼
[E2E Testi (Playwright)]       —> "Kullanıcı tarayıcıda formu doldurup tıkladığında başarı mesajını görüyor mu?"
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: E2E Test Otomasyon Mühendisi
Playwright ve TypeScript kullanarak bir e-ticaret sitesinin 'Kullanıcı Girişi (Login Flow)' için E2E testi yazacağız.

## GEREKSİNİMLER:
1. `http://localhost:3000/login` sayfasına git.
2. E-posta ve şifre inputlarını doldur.
3. 'Giriş Yap' butonuna tıkla.
4. Başarılı girişte `/dashboard` sayfasına yönlendirildiğini ve sayfada 'Hoş geldin Ali' yazısının belirdiğini doğrula.
5. Hatalı şifre girildiğinde ekranda beliren kırmızı hata kutusunu test et.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
// Kötü E2E Test: Kırılgan CSS seçicileri ve sabit bekleme (sleep) tuzağı
test("login naive", async ({ page }) => {
  await page.goto("http://localhost:3000/login");
  await page.click(".btn-primary-2"); // Kırılgan CSS! Buton stili değişirse test çöker!
  await page.waitForTimeout(5000); // KÖTÜ: 5 saniye boşuna beklemek testi yavaşlatır!
});
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
import { test, expect } from "@playwright/test";

test.describe("Kullanıcı Giriş Akışı (Login Flow)", () => {
  test.beforeEach(async ({ page }) => {
    await page.goto("/login");
  });

  test("geçerli bilgilerle giriş yapıldığında dashboard'a yönlendirmelidir", async
({ page }) => {
    // Erişilebilir (Accessible) Rol ve Label Seçicileri (Sağlam & Kırılmaz)
    await page.getByLabel("E-Posta Adresi").fill("ali@example.com");
    await page.getByLabel("Şifre").fill("GuvenliSifre123!");
    await page.getByRole("button", { name: "Giriş Yap" }).click();

    // Otomatik Bekleme (Auto-waiting): URL ve içerik gelene kadar dinamik bekler
    await expect(page).toHaveURL(/.*dashboard/);
    await expect(page.getByRole("heading", { name: "Hoş Geldin
Ali" })).toBeVisible();
  });

  test("hatalı şifrede açıklayıcı hata mesajı göstermelidir", async ({ page }) => {
    await page.getByLabel("E-Posta Adresi").fill("ali@example.com");
    await page.getByLabel("Şifre").fill("YanlisSifre!");
    await page.getByRole("button", { name: "Giriş Yap" }).click();

    const alert = page.getByRole("alert");
    await expect(alert).toBeVisible();
    await expect(alert).toContainText("Geçersiz e-posta veya şifre.");
  });
});
```

💡 **Kod Analizi:** `getByLabel` ve `getByRole` gibi erişilebilirlik (ARIA) seçicileri kullanılarak UI değişikliklerine karşı dayanıklılık sağlanmış; `page.waitForTimeout` yerine Playwright'ın dahili 'Auto-Waiting' özelliğiyle testler maksimum hızda çalıştırılmıştır.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Testlerde `page.waitForTimeout(3000)` (sabit uyku) kullanıp test koşu sürelerini uzatmak.
  - **Çözüm:** Sabit sleep yerine Playwright'ın otomatik bekleyen `expect(locator).toBeVisible()` metodlarını kullanın.
- **Tuzak:** Kırılgan CSS seçicileri (`div > div.col-md-4 > button`) kullanmak.
  - **Çözüm:** Daima kullanıcı odaklı ARIA seçicileri (`getByRole` , `getByText`) veya `data-testid` kullanın.
- **Tuzak:** Her küçük buton için E2E testi yazıp CI pipeline süresini 45 dakikaya çıkarmak.
  - **Çözüm:** E2E testlerini sadece kritik kullanıcı akışları için yazın; detayları Unit testlere bırakın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 33: Uçtan Uca Testler - Playwright ile Tarayıcı Otomasyonu*

- 🖥️ **Slayt 1:** Unit testlerin göremediği entegrasyon kör noktaları.
- 🖥️ **Slayt 2:** Playwright mimarisi: Çoklu tarayıcı (Chromium, WebKit, Firefox).
- 🖥️ **Slayt 3:** Kırılmaz seçiciler: ARIA Rolleri ve data-testid kullanımı.
- 🖥️ **Slayt 4:** Canlı Demo: Playwright UI Mode ve Trace Viewer ile hata ayıklama.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Terminalde `npx playwright test --ui` komutunu çalıştırın. Playwright UI penceresinde testin tarayıcıyı nasıl açıp formu doldurduğunu, zaman çizelgesinde (timeline) her adımın ekran görüntüsünü nasıl kaydettiğini canlı gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenize Playwright ekleyin (`npm init playwright@latest`) ve ana sayfanızın açılıp başlığının doğru göründüğünü doğrulayan ilk E2E testinizi çalıştırın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: E2E testlerinde `page.locator('button.btn-blue-submit')` yerine `page.getByRole('button', { name: 'Giriş Yap' })` tercih edilmesinin sebebi nedir? - A) Daha az harf içermesi - B) CSS sınıfları değişse bile kullanıcının gördüğü metin ve erişilebilirlik (ARIA) rolü değişmediği sürece testin kırılmadan çalışmaya devam etmesi - C) Sadece Safari'de çalışması - D) Bilgisayarın sesini açması > **Doğru Yanıt:** B > **Açıklama:** Kullanıcı odaklı (User-Facing) seçiciler, frontend tasarım revizyonlarından etkilenmez ve testleri son derece sağlam (resilient) kılar.

Soru 2: Playwright'ın 'Trace Viewer' aracının yazılımcılara sağladığı en büyük süper güç nedir? - A) Bedava kahve ısmarlaması - B) Testin koştuğu her milisaniyenin ekran görüntüsünü, konsol loglarını ve ağ (network) isteklerini zaman makinesi gibi geriye sararak inceleme imkanı vermesi - C) Kodları otomatik silmesi - D) Sadece Windows'ta çalışması > **Doğru Yanıt:** B > **Açıklama:** Trace Viewer, özellikle CI ortamında çöken E2E testlerinin tam olarak hangi saniyede ve hangi ağ hatasıyla patladığını video gibi incelemeyi sağlar.